Decisions

How are predictions used to make decisions that provide the proposed value to the end-user?

- Real-time GUI displays part name, confidence, and image.
- Used to:
  - classify parts according to shape, colour.
  - Assist users in building symbol-to-brick associations.

ML task

Input, output to predict, type of problem.

- **Task Type:** Supervised learning.
- **Problem:** Multi-class real time classification .
- **Input:** LEGO part image / webcam input.
- **Output:** LEGO part class + colour tag.

Value Propositions

What are we trying to do for the end-user(s) of the predictive system? What objectives are we serving?

- **Automated Sorting & Storage System**: LEXIS classifies and organizes LEGO pieces in real-time.
- **Tiny Piece Recognition**: Excels at recognizing **very small and ambiguous parts** that are time consuming to identify manually or with traditional systems.
- **Dual Classification**: Classifies both **part type** and **colour** for advanced cataloguing or recycling.
- **Sustainability**: Aids in recycling and repurposing LEGO bricks by identifying pieces for reuse.
- **Symbol & Shape Learning**: Supports educational use by helping users learn part IDs, and symbols through interaction.
- **Supports Real-Time & On-Demand** Predictions via webcam or mobile camera.

Data Sources

Which raw data sources can we use (internal and external)?

Internal:

- 46,000+ hand-labelled real-life LEGO part images across 23 classes.
- 5,200 synthetic 3rd generated data of LEGO parts for 7 classes.
- 1,100 yolo LEGO labelled images.
- Colour data (manual + image-derived) for RGB-based colour tagging.

External:

- Pretrained MobileNetV2 (from TensorFlow).[trained on +1M datapoints and have over 1000 classes]

Collecting Data

How do we get new data to learn from (inputs and outputs)?

Inputs:

- **Images**:
  - Captured from a webcam feed
  - Uploaded by users (as a dataset)

Outputs:

- Labeled Image
- predicted class
- confidence score

Making Predictions

When do we make predictions on new inputs? How long do we have to featurize a new input and make a prediction?

When Triggered:

- Webcam/Mobile stream capture.
- Manual user upload.

Training time depends on device configurations and processing power

Fallback Logic:

- If YOLOxCNN confidence < 30% or if class is in “hard-to-detect” list → fallback to MobileNetV2.
- Colour prediction runs post-classification.

Offline Evaluation

Methods and metrics to evaluate the system before deployment.

Pre-Deployment Testing:

- Train/Validation/Test split with stratified sampling.
- Evaluate:
  - Accuracy, Precision, Recall, F1-Score
  - Confusion Matrix
  - Per-class performance

Tools Used:

- TensorFlow
- scikit-learn
- matplotlib

Features

Input representations extracted from raw data sources.

- **Input:** RGB images resized to 96×96 pixels.
- **Preprocessing:** Resize, normalize, optionally augment.
- **Features Extracted:**
  - CNN embeddings (MobileNetV2).
  - Object bounding boxes (YOLO).
  - Colour histograms or centroid RGBs for secondary colour prediction.

Building Models

When do we create/update models with new training data? How long do we have to featurize training inputs and create a model?

Model Triggers:

- New LEGO parts added.
- Feedback loop detects performance decline.
- New usage context (e.g., different lighting, camera, or colours).
- Transfer learning to strengthen or to add new classes

Training Time:

- ~ 4–6 hours on CPU (MobileNetV2 transfer learning). (depends)
- YOLO fine-tuning possible with new bounding boxes.

Live Evaluation and Monitoring

Methods and metrics to evaluate the system after deployment, and to quantify value creation.

- Detection confidence (YOLO + CNN)
- Piece count & label frequency
- Fallback usage when YOLO fails
- Autosaved Excel logs every 30s
- Colour accuracy matched to LEGO palette.